

Arcade Suite-

Bachelorpraktikum

Project Documentation

Artjom Becker, Ben Klasen, Denis Shamrin, Djamil Barth, Lars Zell

February 28, 2025



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Clients: Jannis Blueml,
Quentin Delfosse

1 Project description

Our project focuses on Atari games and their use as a playground for various machine learning approaches. Due to their simple structure, Atari games provide an ideal foundation for testing and improving different reinforcement learning (RL) methods. In this context, we joined an open-source project that follows an innovative approach to objectification in these games.

The goal of this approach is to enable agents to identify and recognize different objects within a game. For the game Pong, it would be *Player*, *Enemy*, *Ball*. There are two methods for capturing these objects:

1. **RAM extraction method:** Relevant object characteristics are directly read from the RAM state
2. **Vision extraction method:** In this method, object detection is performed through image recognition and visual processing

The primary goal is for the developed Object-Centric Agents to make more conscious and robust decisions in the game. To test this, a subclass called Hack Atari was developed, which allows modifications to be made to different Atari games. These modifications include limiting boxing to using only one hand in the game Boxing, changing the colors of player entities, or varying the speeds. Such modifications often pose no challenge for humans, but they can challenge certain algorithms, especially deep learning approaches. Changing the Color of the Ball in Pong wouldn't do much to a human playing it, while some DL Agents don't work anymore at all.

An agent in this project receives the current state of the game as input and then selects an appropriate action, such as *Move left*, *Move right*, or *Fire*. How this input is processed depends on the RL method used.

For the OC agents, decision trees can be generated, which allows the reasoning behind an action to be traced. In these trees, relationships between objects are considered and compared with a threshold that was determined through the system's learning process.

Our task was to visualize these decision trees for the OC agents. The goal was to make visible for every game state which object states and relationships are crucial for the agent to make a particular decision. This was to be implemented on a web platform where users could choose between different Atari games and make modifications to the game environment. The changes and modifications would then be visible within the respective game.

Finally, the challenge was to develop a multiplayer system that allows players to compete either against agents or other players. Additionally, the agents should also be able to play against each other to test their performance in different game scenarios. Because the current HackAtari doesn't support Multiplayer, our System won't conclude any form of Modification in this mode.

Our project makes an important contribution to understanding how OC agents make decisions. While deep learning algorithms are often susceptible to changes in the game environment, such as color changes or variations in speed, the OC approach shows that agents are able to respond more flexibly and adaptively to changes. By visualizing object relationships and decision-making processes, it becomes possible to better understand why an agent has chosen a specific action. This is especially important for understanding and promoting the generalized behavior of agents – that is, the ability of agents to transfer their experiences from one game to new, unknown game situations. Especially in the current times, where the introduction of self-driving cars is being discussed, it is important that such systems are robust against any kind of change.

2 Developing Process

2.1 Requirements management

During the development phase of our project, we collected, managed, and prioritized requirements. We used the tool Trello to maintain a clear overview and efficiently organize our tasks. The requirements were divided into two categories: fundamental requirements, which served our Minimum Viable Product (MVP) and defined goals, and additional features that provided extended functionalities. An example of the latter is the implementation of a heatmap to visualize action-relevant objects.

We categorized our requirements into three effort levels to effectively distribute them among our group:

- **Low effort:** Implementation takes about one day
- **Medium effort:** Completion within half an iteration
- **High effort:** Can require multiple iterations

This assessment was carried out collectively within our team, based on the estimated complexity and our experience with similar tasks.

We gathered requirements during regular meetings with our clients and subsequently distributed them. During these meetings, existing requirements were reviewed and updated if necessary. Especially at the beginning of the project, many requirements involved understanding the existing system and making it work on our own systems. A key aspect was the ability to run an agent in a modified Atari environment. Each team member was required to test this in order to develop a shared understanding of the system structure.

In retrospective, we underestimated the effort required to familiarize ourselves with existing structures, particularly distributed projects and packages. The significant time investment needed to comprehend these constructs was often miscalculated, leading to delays.

One example of this was the decision to choose a suitable framework for the web implementation. To make an informed decision, we needed to fully understand the challenges ahead. A crucial requirement was integrating an existing Atari game-running in a Pygame frame- into our web application. Ultimately, our chosen framework proved impractical, as the integration turned out to be more challenging than expected. Additionally, the documentation for the selected framework was not very helpful and had to be collected from various sources. Since our chosen framework, NiceGUI, is based on Quasar, we were also forced to familiarize ourselves with that.

Once the initial challenges were overcome, we moved on to more practical requirements, such as developing a menu structure and creating initial concepts and illustrations for the decision tree (Figure 1). The requirements were distributed within the group in a way that allowed them to build upon one another, ensuring a meaningful subdivision of the project structure (More on this in the Organization section).

At the beginning, we struggled to productively engage in the development process. One reason for this was that some conceptual questions remained unclear. Since we were initially uncertain to ask our client for clarification directly, some progress was delayed. Over time, however, communication became more active, which positively impacted our development.

In the early stages, the scope of completed requirements was relatively small, but this changed over time. Around the Christmas season, the initial menu structure brought momentum to the project. Motivated by small successes and a growing team spirit, our development became increasingly more efficient.

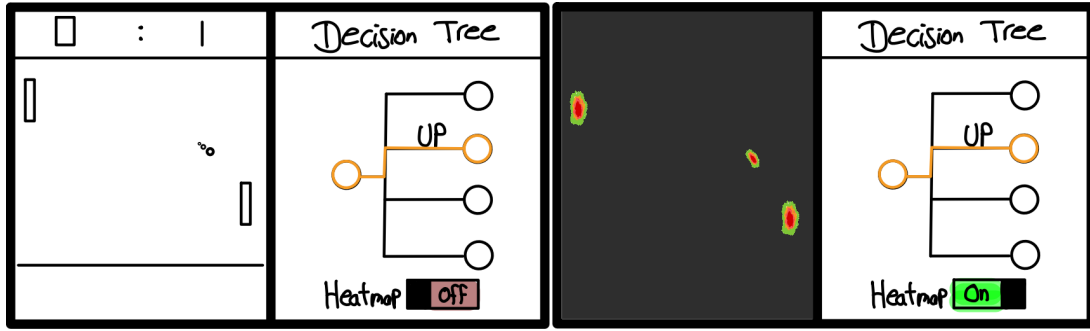


Figure 1: Initial Illustration for the Decision Tree and Heatmap. In the Beginning the implementation of a heatmap was still under consideration, but it ultimately was not part of our MVP.

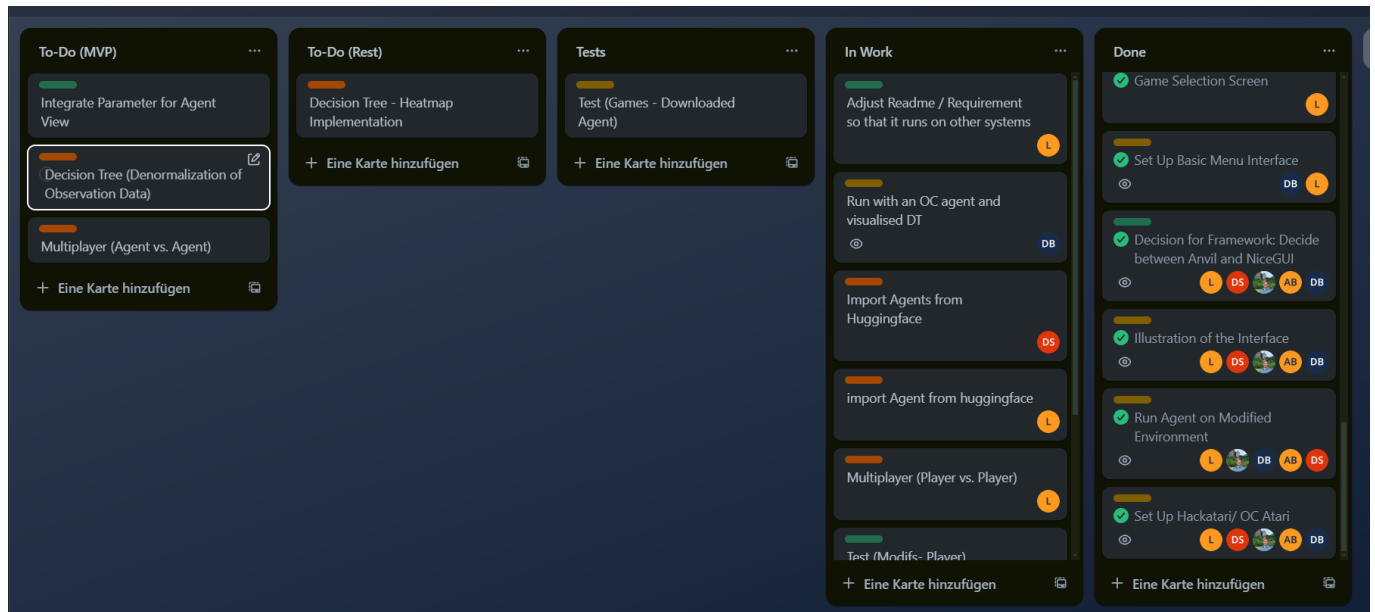


Figure 2: An insight into our requirements on our Trello board.

2.2 Quality assurance

The quality assurance of our project proved to be challenging, particularly due to the development of an application with a graphical user interface. This normally required testing both the backend and the frontend. Since the requirements for the GUI were not precisely defined, our primary focus was on backend testing.

Our main task was to ensure the functionality of modified Atari games as well as the integration of agents/players. This led to following testing approaches:

1. **Exhaustive testing of Atari games:** Since the test space was limited by the number of available games, we decided to achieve full test coverage for these games. This strategy allowed us to establish a fixed pool of working Atari games.
2. **Random sampling tests for modifications:** Due to the increased complexity introduced by modifications, exhaustive testing was not feasible. Therefore, we conducted random sampling tests.

Regarding the sampling tests for modifications, we differentiated between two sources of errors:

- **Incomplete implementation of the modification:** The modification had not yet been fully or correctly implemented in the HackAtari wrapper. An error is thrown during the attempted execution.
- **Errors in triggering the modifications:** Existing modification structures were incorrectly invoked. If a modification is accessed correctly, the RAM state will be changed with respect to certain parameters.

Since only the second error source fell within our area of responsibility, we focused on addressing those errors. We kept a list of modifications that threw an error message to disable these modifications until their functionality was ensured in HackAtari. The

implementation of agents also introduced a high level of complexity. To manage this, we defined test cases for various machine learning approaches:

1. **DQN (Deep Q-Networks)**
2. **PPO (Proximal Policy Optimization)**
3. **OC (Object Centric Agents)**
4. **C51 (Distributional RL)**

So we made sure that different types of agents were supported. OC agents were particularly relevant in testing, as they were supposed to be displayed using a decision tree representation. Within our repository, we maintained a documented list of functional Game modes. If an error affected an entire agent class, we attempted to develop a general solution in exchange with our clients. For our Tests we used the Framework pytest.

Certain aspects of the multiplayer implementation are still incomplete. Player vs. Player functionality is already available, but any multiplayer mode involving an agent has not yet been implemented. This is due to differences between the environment in multiplayer and single-player modes. The environment states are represented differently and are not currently recognized by our agents. We are in communication with our client to resolve this issue. While it is not a major problem, it is expected to be addressed next week, as our client currently has a lot of work to do.

In addition to functional testing, code reviews played a crucial role in our quality assurance process. The main principle was that every piece of code had to be reviewed by at least two team members and then discussed. We conducted these reviews through Discord calls, involving both the reviewers and the original author of the code. These discussions proved invaluable, as they allowed us to gather diverse perspectives and exchange opinions and improvement suggestions regarding code styling, data structures, and libraries used.

Due to the high effort involved in these reviews, we had to limit their scope at times to stay within our timeframe. Nevertheless, the code reviews led to important optimizations:

- **Code readability:** Improved through consistent naming conventions and inline comments
- **Refactoring:** Enhanced generalization of functionalities → Particularly in the GUI area, we reduced redundant elements through inheritance
- **Data structure optimization:** Initially, a list was planned for storing game modifications, but a dictionary proved to be more advantageous (also improving usability)
 - **Key-Value:** Textual representation of a modification in our GUI
 - **Data Value:** Internal processing and invocation of the HackAtari structure

During development, several preview and simulation features were created, which were not needed in the final version. However, these features were useful for validating the system and detecting potential errors early on.

Throughout the project, pair programming was not a primary focus, as tasks were generally worked on independently. However, at the beginning of the project, some team members worked in parallel on the same problem areas, which facilitated an intensive exchange of ideas regarding potential solutions and challenges. As the project progressed, we primarily collaborated in co-working sessions, discussing specific issues as needed. Additionally, we held regular team meetings to review progress, address open questions, and make decisions on complex features together.

Code Readability and Structure			
Question	Pass	Filled	Comment
Code is well structured ?			
Style is consistent ?			
Are there complex Structured which need be restructured or split into multiple Routines ?			
Has every Class one specific purpose ?			
Class/Method/Attribute names are choosen well ?			
Datastructures are optimal for usecase ?			
Code is clearly and adequately documented ?			

Functunality			
Question	Pass	Filled	Comment
Are there any Uncalled / Unneeded Procecedures ?			
Can any Code be replaced by calls to external reusable components or library functions ?			
Are there any Error Handlings ?			
Are Async Functions used the right way ?			

Figure 3: Our Code-Review-checklist that we created as a group after the choice of the web framework was made.

2.3 Organization (Version control)

Structured and efficient communication, as well as well-thought-out organization, were essential for the success of our project. Our development process followed a two-week iteration cycle, which was accompanied by regular internal and external meetings.

Our development process was divided into three main types of meetings:

1. Debrief after the client meeting

- Directly following the meeting with the clients, our first internal meeting took place
- **Goal:** Review and distribution of requirements, assigning initial tasks (applied to Trello)
- Important information, dates, and deadlines were documented in a dedicated Discord channel

2. Mid-iteration meeting (halfway through the iteration)

- Weekly exchange about the current development status
- **Goal:** Identification and discussion of issues or bottlenecks
- **Actions:** Re-prioritization or redistribution of tasks

3. Preparation meeting for the client meeting

- A few days before the next client meeting, a final alignment took place
- **Goal:** Summary of current developments and changes for presentation

The meetings with the clients initially took place in person to get to know the Arcade Machine but were later conducted entirely remotely via Zoom. Communication with our clients was conducted via the Mattermost platform. Our clients were almost always available, and requests were typically answered within a day. Unfortunately, we initially neglected this communication option, leaving a significant optimization potential untapped.

For internal communication and documentation, we used a Discord channel, where we centrally collected important information such as documentation, installation instructions, and organizational notes. Regular communication within the team helped avoid

misunderstandings and kept us on track with progress.

Our project consisted of three core components:

1. **GUI application for displaying the game, agent, and modifications**
2. **Decision Tree implementation**
3. **Multiplayer implementation**

To keep the development modular, we utilized Git branching. Each main task was assigned its own branch to enable parallel development. Since the GUI application was the heart of our project, it formed the master branch. Within the group, we were assigned fixed branches, allowing us to build a more comprehensive understanding of a problem while also working in parallel and modularly. Requirements that built upon each other could be processed more efficiently this way.

2.4 Dealing with challenges

Throughout the development of the project, various challenges arose that required adjustments to our approach. On a positive note, the project phase proceeded without any significant illness-related absences. However, unexpected difficulties did impact our development process.

One central challenge was changes to the base construct, particularly modifications to HackAtari. These changes rendered our previously chosen adaptation strategy incompatible, as the internal handling of modification names had been altered. This forced us to make retrospective adjustments, leading to significant additional effort. In hindsight, an early analysis of the wrapper changes would have helped avoid unnecessary implementation work.

Furthermore, formulating and refining requirements posed an additional challenge. Specifically, the development of the Decision Tree required an efficient representation that allowed for continuous updates during gameplay. Due to the tree's complexity and the inefficient re-rendering strategy, implementation became extremely time-consuming. During meetings, new ideas continuously emerged for improving the representation of the Decision Tree. However, we spent considerable time working on details that were less relevant to the client, while essential requirements were not implemented in a timely manner. A core issue was that the requirements for the Decision Tree were defined very early in the project phase, even before we fully understood the scope and complexity of the task. This led to misunderstandings in communication. Our mistake was not recognizing this discrepancy sooner and failing to address it earlier. After intensive coordination with the client, a more compact, yet detailed representation was agreed upon, which could be implemented more efficiently. Despite the ultimately pragmatic solution, the effort invested earlier was substantial.

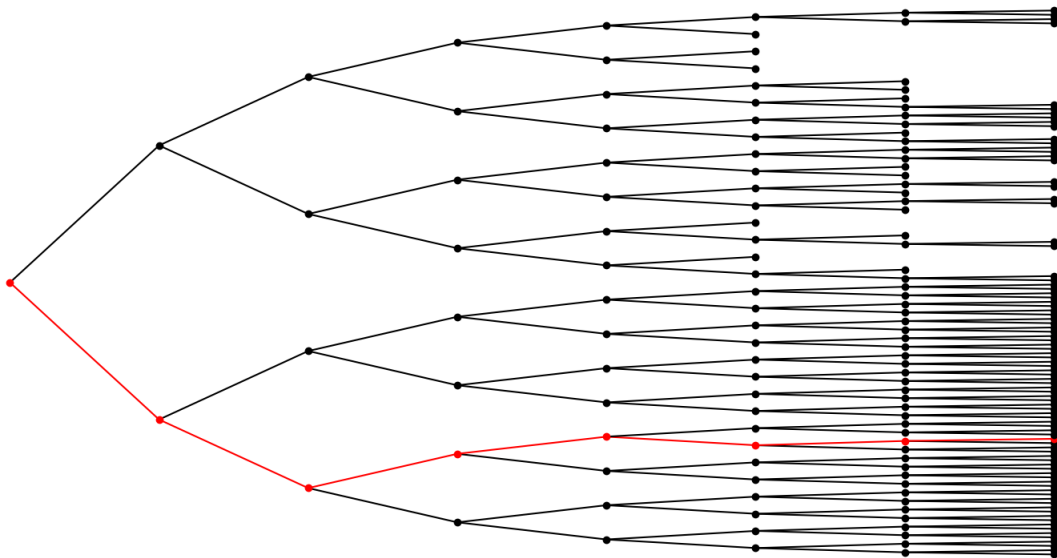


Figure 4: First Representation of the Decision Tree (Bad Runtime)

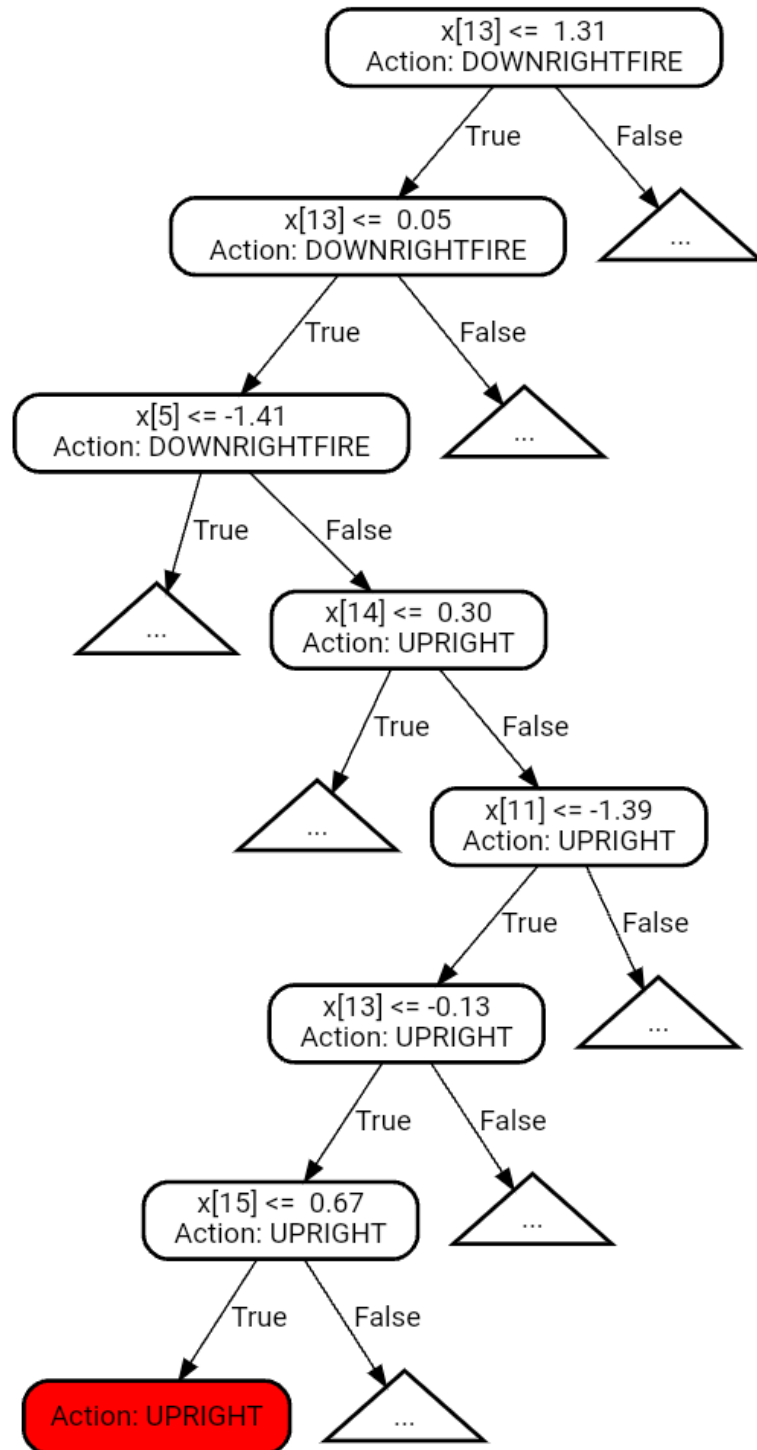


Figure 5: Second Representation of the Decision Tree (Focus on Path with more information)

```
Player1.x(-5.09)  <=  0.01
      D(Player1, Flag1).x(-3.54)  <=  0.28
            Flag2.y(-4.91)  <=  -1.15
                  Tree2.x(-9.74)  <=  0.53

Chosen Action:  FIRE

Feature Evaluation:
      -∞ <=  Player1.x  <  0.01
      -∞ <=  D(Player1, Flag1).x <  0.28
      -∞ <=  Flag2.y  < -1.15
      -∞ <=  Tree2.x  <  0.53
```

Figure 6: Final Representation of the Decisiontree (More Information and Understanding about the actions of the Agent)

3 Conclusion

The project was an educational and exciting experience that challenged us both technically and organizationally. One of the biggest hurdles was understanding existing structures and frameworks. At the beginning, we underestimated the effort required to grasp the existing systems and integrate them into our own development. This led to delays, but by continuously improving our communication(both within the team and with the clients) we were able to overcome these obstacles.

A key aspect of the process was effective requirements management. By using Trello and holding regular meetings, we were able to prioritize our tasks efficiently. However, we realized that defining certain requirements too early—especially for the decision tree visualization—was problematic. The initial concepts turned out to be too complex, requiring later adjustments. This taught us that an iterative approach with regular feedback is essential to avoid misunderstandings and inefficient development.

The technical implementation also presented challenges, particularly due to unexpected changes in HackAtari and the integration of the chosen framework. While we initially struggled with documentation and architectural complexity, consistent testing and regular code reviews helped us maintain a stable and understandable development process. These measures not only improved code quality but also deepened our understanding of the individual components.

Despite the challenges, working on this project was a lot of fun. Successfully implementing features and seeing visible progress motivated us, especially in the later stages. Our teamwork also evolved positively: while there were some uncertainties at the beginning, we became increasingly efficient over time. Collaborative debugging sessions and discussions about possible solutions proved to be extremely valuable.

In conclusion, we learned a lot about teamwork, iterative development, and handling unexpected problems. In future projects, we would adopt a more flexible approach to defining requirements from the start and emphasize regular feedback even more. At the same time, we would retain many of our methods, such as structured testing and close team collaboration. Overall, we are proud of what we accomplished and take away many valuable experiences.

4 Github Repository

<https://github.com/DjamilB/ArcadeSuite>

5 Appendix

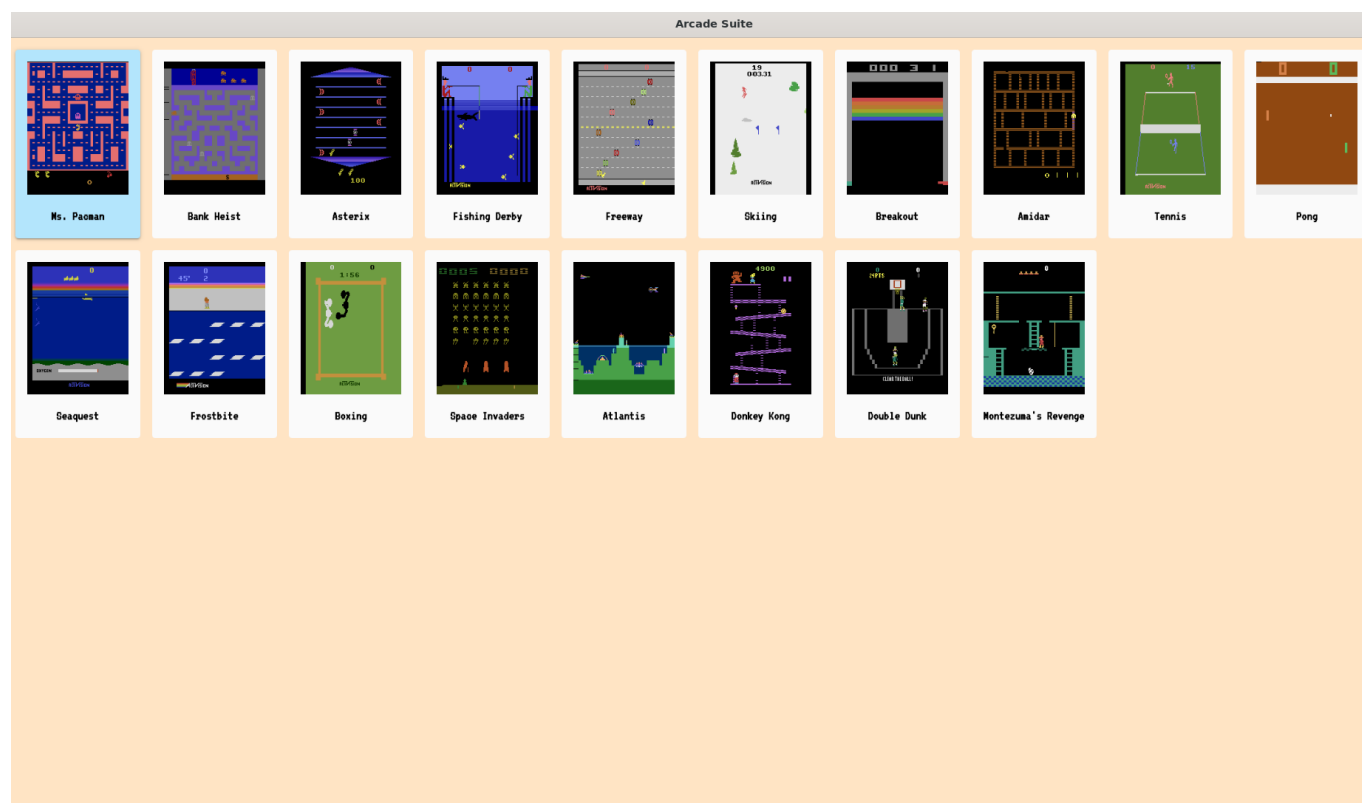


Figure 7: Current Menu View

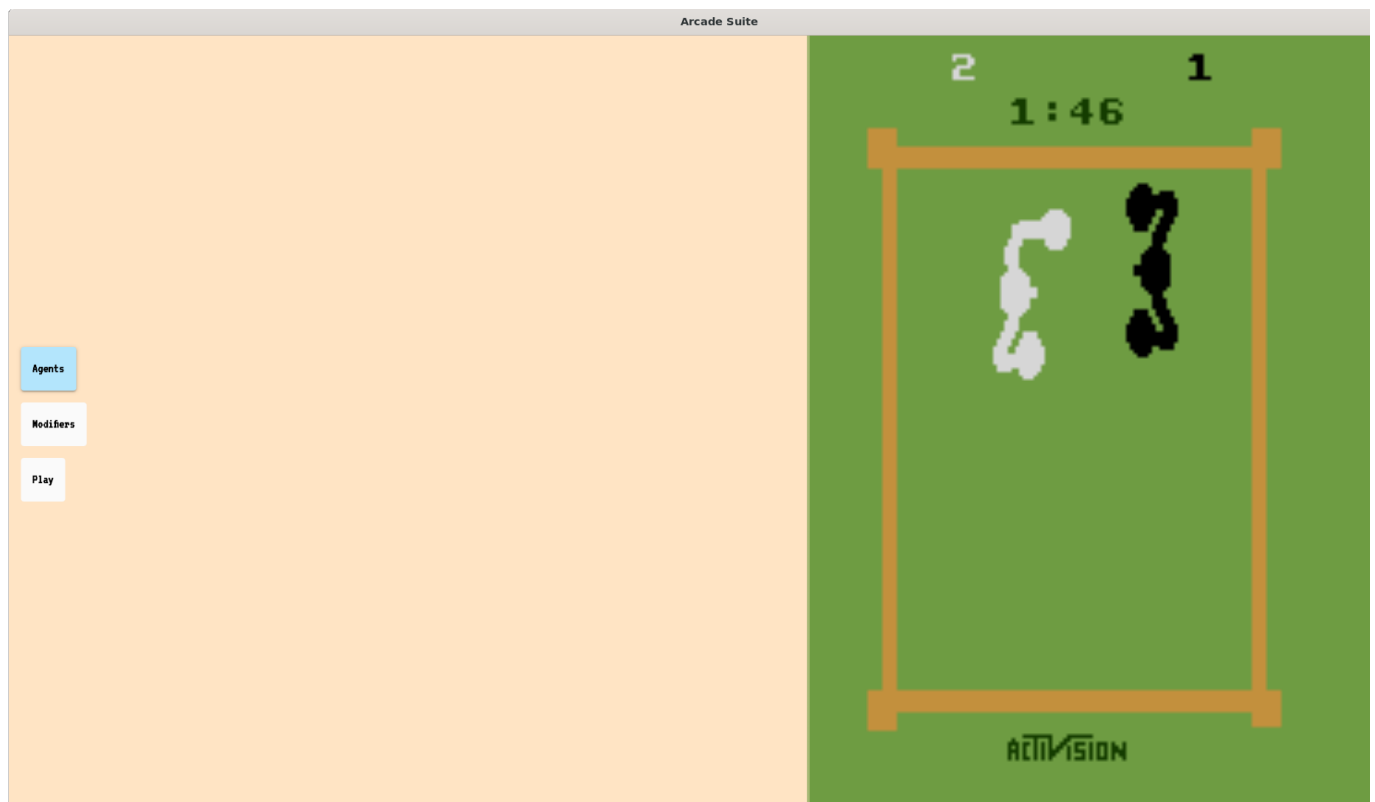


Figure 8: Current Selection View

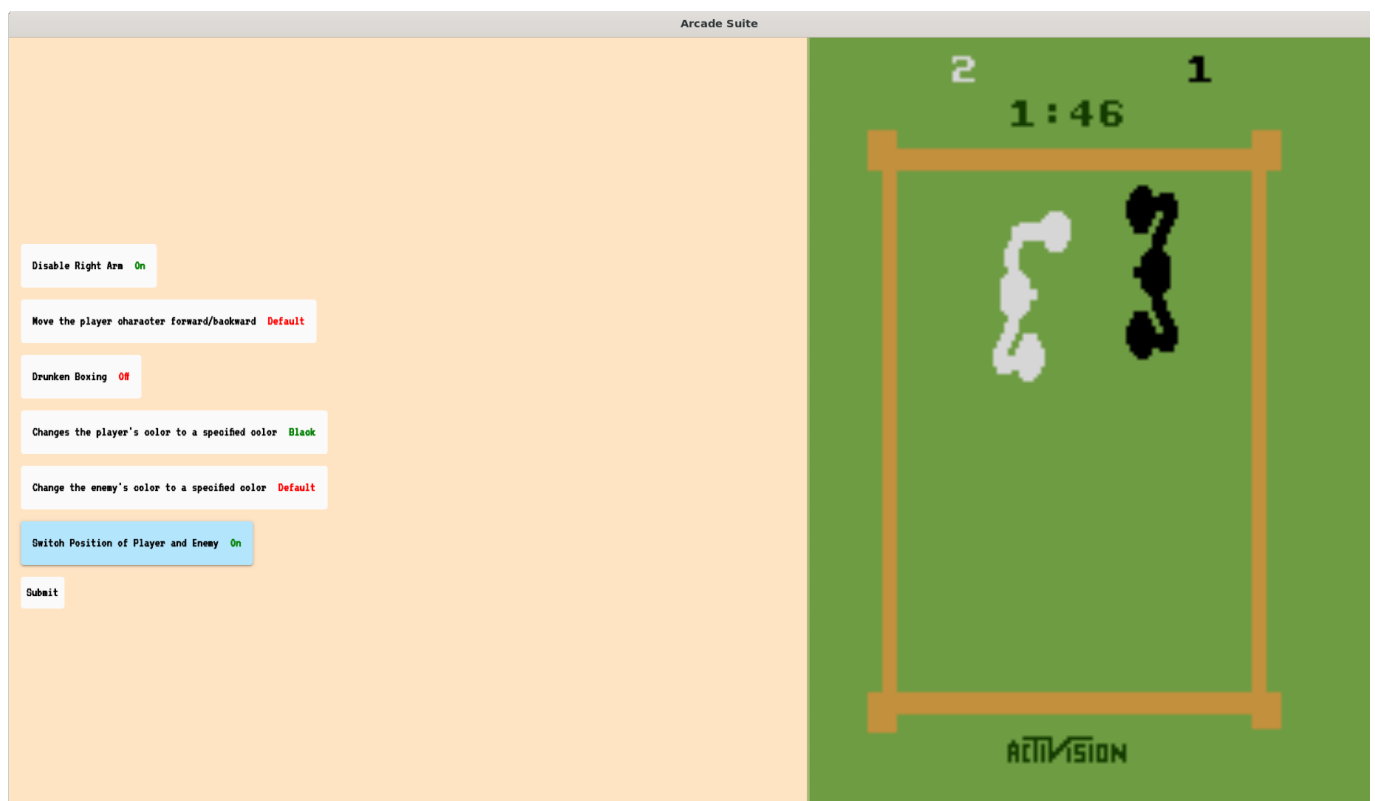


Figure 9: Current Modification Selection View

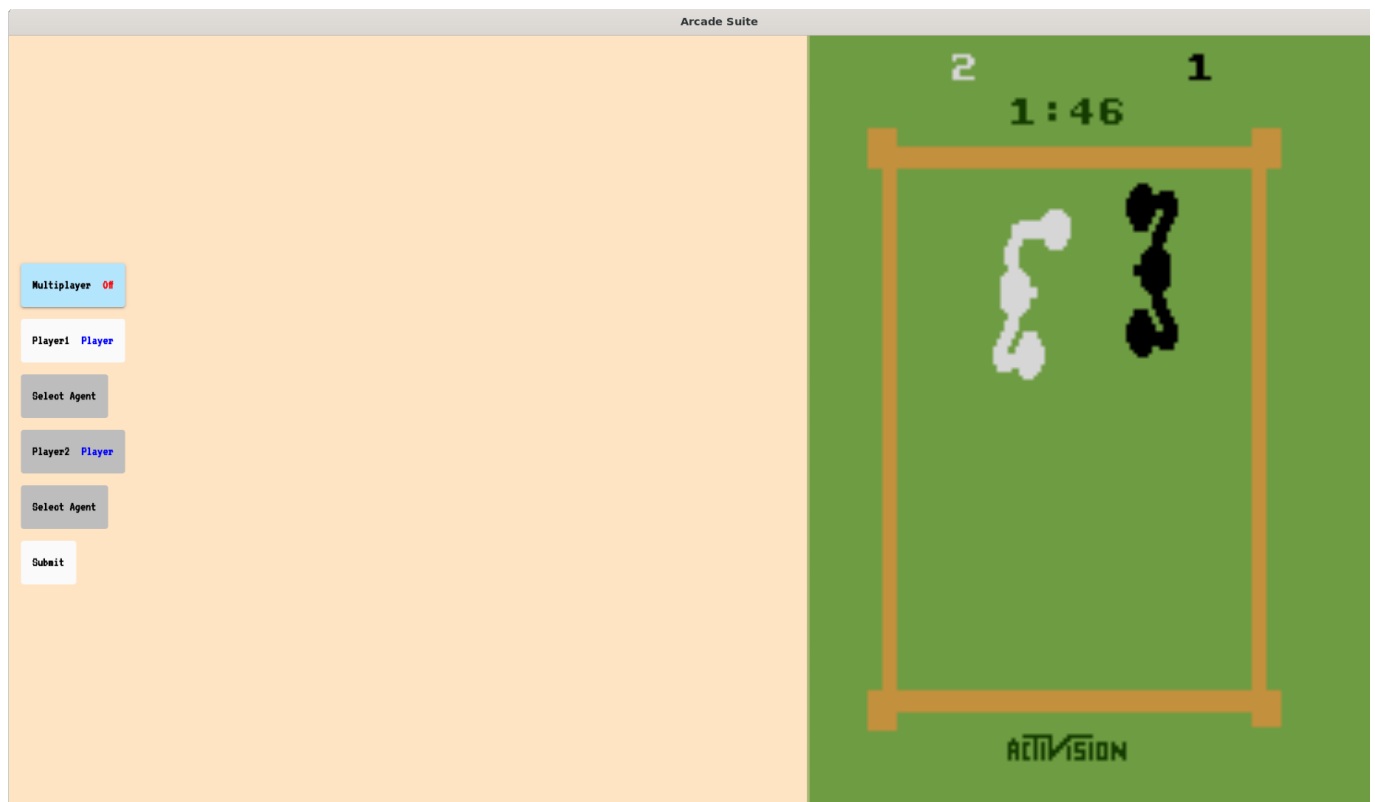


Figure 10: Current Player Selection View

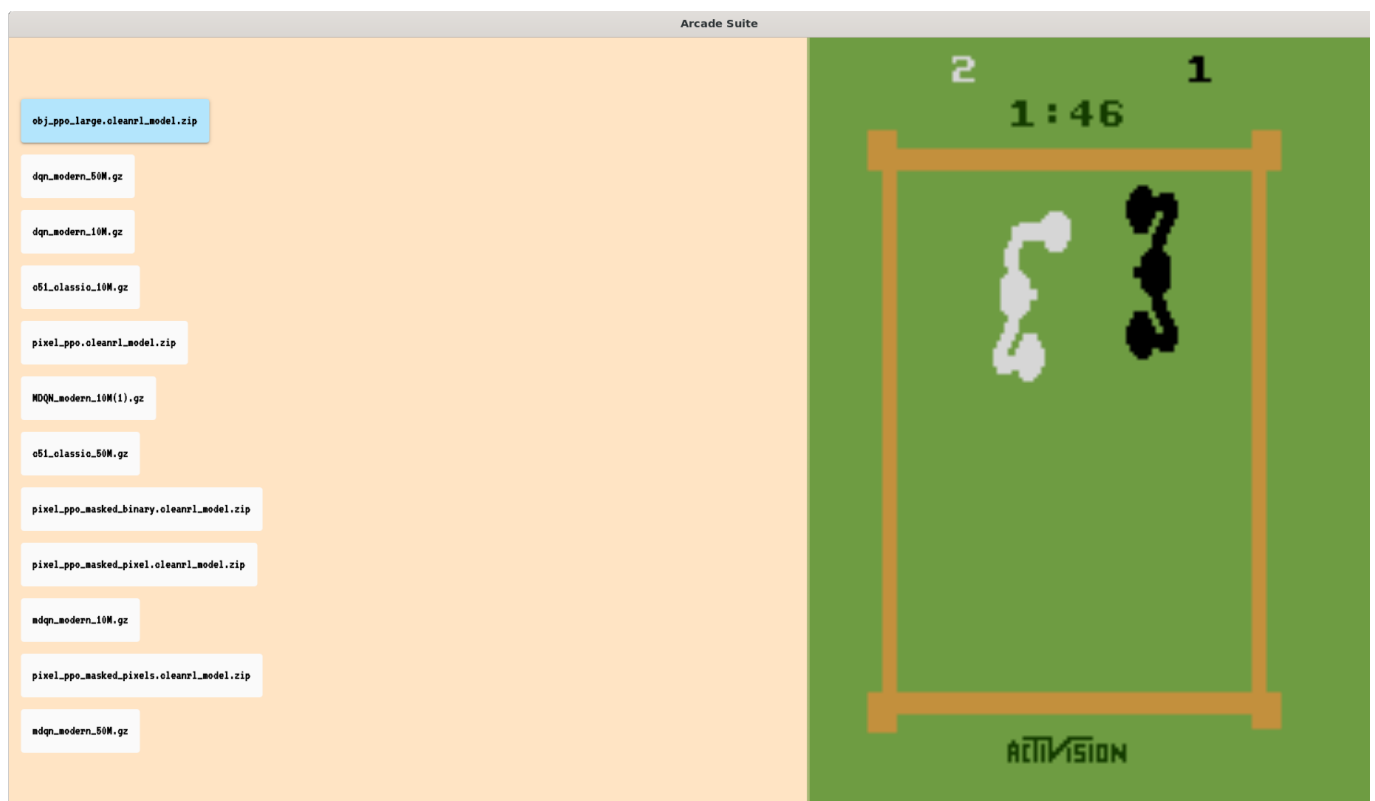


Figure 11: Current Agent Selection View: Agents will be sorted by approach in future

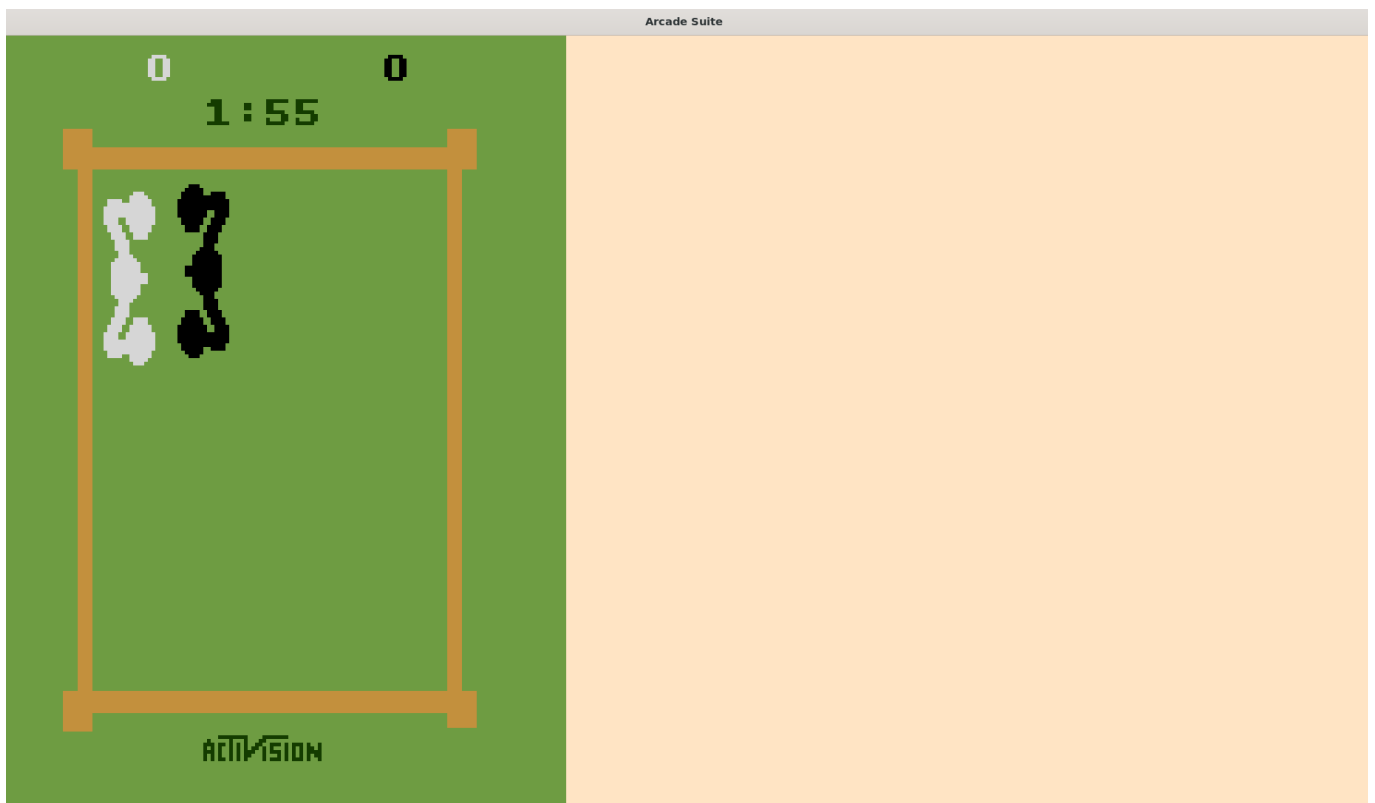


Figure 12: Current Game View: Unfortunately, the OC agents that are to be displayed with a decision tree are not yet working. We are in contact with our clients in this regard, who are still sending us the interface (as of 27/02/2025)